

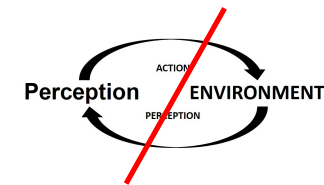


Artificial Intelligence

Solving Problems by Searching
Ch 3

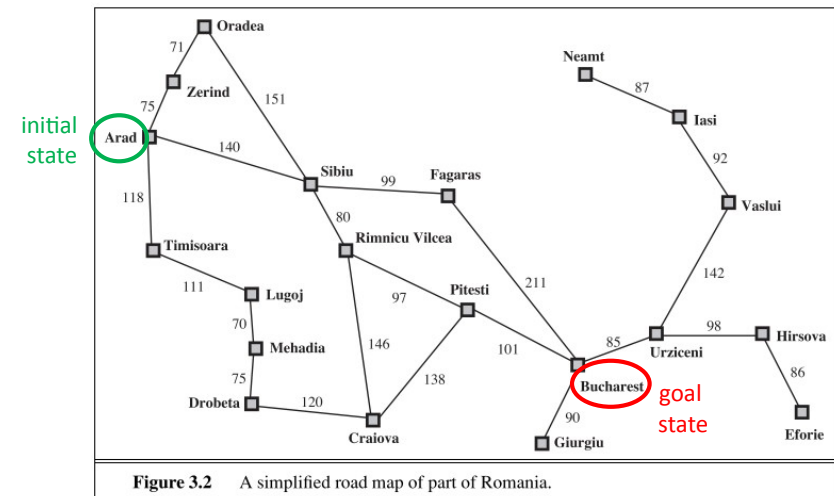
Problem-Solving Agents

- Goal-based agent.
- Task: find out a **sequence of actions** that leads to a goal state (problem solution). This is called **search**.
- Open-loop design: formulate, search, execute.



Environment

- **Observable**, so the agent always knows the current state (e.g., the city the agent is currently visiting).
- **Discrete**, so at any given state there are only finitely many actions to choose from (e.g., the cities the agent can reach from the current one).
- **Known**, so the agent knows which states are reached by each action (having an accurate map suffices to meet this condition for navigation problems).
- **Deterministic**, so each action has exactly one outcome. It means that if the agent chooses to drive from City A to City B, it does end up in City B.
- **Atomic representations**: states of the world are considered as wholes



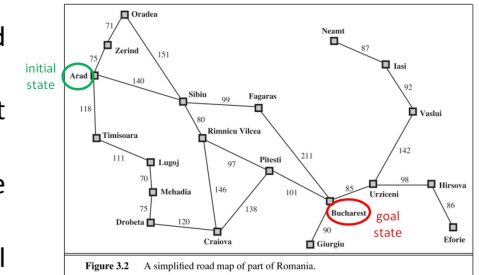
Well-defined Search Problems

A problem can be defined formally by five components:

1. **Initial state.**
2. **Possible actions** available **in each state**.
3. **Transition model:** a function that returns the state that results from doing action a in state s (**successor**).
4. **Goal test:** function that determines whether a given state is a goal state.
5. **Path cost function:** assigns a numeric cost to each path.
 - The **step cost** of taking action a in state s to reach state s' is denoted by $c(s, a, s')$. We assume that step costs are nonnegative.

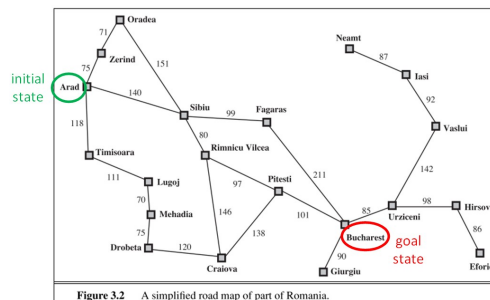
Route-Finding Problem

- **States:** Locations.
- **Initial State:** Initial location specified by the user.
- **Actions:** Destination from the current location.
- **Transition model:** Arrival at the destination.
- **Goal test:** Are we at the final destination specified by the user?
- **Path cost:** sum distances in kilometers.



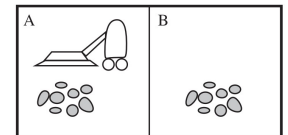
Problem State Space

- Defined by Initial state + Actions + Transition model
- Directed graph:
 - Nodes are states
 - Links are actions.
- **Path:** sequence of states connected by a sequence of actions.

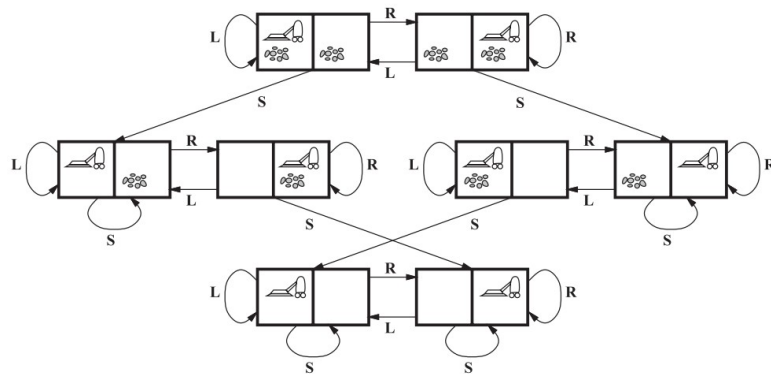


Toy Problem 1: The Vacuum World

- **States:** Agent location and Booleans indicating if there are dirt in each cell.
 - $2 \times 2^2 = 8$ possible world states.
- **Initial state:** Any state can be designated as the initial state.
- **Goal test:** This checks whether all the squares are clean.
- **Actions:** Left, Right, and Suck.
- **Transition model:** Actions have their expected effects, except that moving Left in the leftmost square, moving Right in the rightmost square, and Sucking in a clean square have no effect.
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.



The state space for the vacuum world.
Links denote actions: L = Left, R = Right, S = Suck.



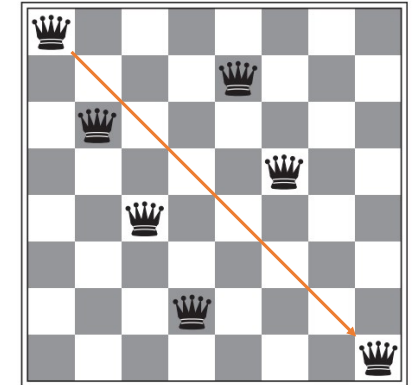
This toy problem has discrete locations, discrete dirt, reliable cleaning, and it never gets any dirtier.

8-Queens Problem

Place eight queens on a chessboard such that no queen attacks any other.

• **Incremental formulation:** each action adds a queen to the state.

• **Complete-state formulation:** starts with all 8 queens on the board and moves them around.



Incremental 8-Queens

- **States:** Any arrangement of 0 to 8 queens on the board is a state.
- **Initial state:** No queens on the board.
- **Actions:** Add a queen to any empty square.
- **Transition model:** Returns the board with a queen added to the specified square.
- **Goal test:** 8 queens are on the board, none attacked.
- **Path cost:** 0 (irrelevant).

In this formulation, we have $64 \cdot 63 \cdot \dots \cdot 57 \approx 1.8 \times 10^{14}$ possible sequences to investigate.

Incremental 8-Queens [better formulation]

Prohibit placing a queen in any square that is already attacked:

- **States:** All possible arrangements of n queens ($0 \leq n \leq 8$), one per column in the leftmost n columns, with no queen attacking another.
- **Actions:** Add a queen to any square in the leftmost empty column such that it is not attacked by any other queen.

This formulation reduces the 8-queens state space from 1.8×10^{14} to just 2.057, and solutions are easy to find.



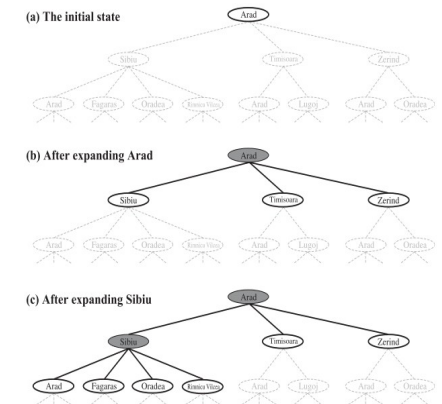
Artificial Intelligence

Searching for Solutions

Ch 3.3

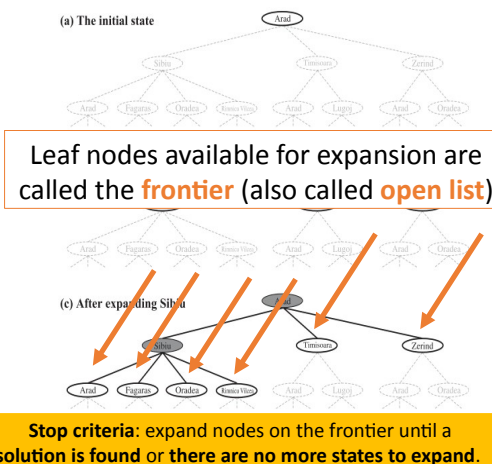
Search Algorithms

- Work by considering **various possible action sequences**.
- **Search tree**: initial state at the root. Branches are actions and non-root nodes correspond to successor states.
- **Expansion**: use actions to **generate** successor states of a node.
- **Essence of search**: follow up a successor now and put the others aside for later, in case the first choice does not lead to a solution.



Search Algorithms

- Work by considering **various possible action sequences**.
- **Search tree**: initial state at the root. Branches are actions and non-root nodes correspond to successor states.
- **Expansion**: use actions to **generate** successor states of a node.
- **Essence of search**: follow up a successor now and put the others aside for later, in case the first choice does not lead to a solution.



function TREE-SEARCH(*problem*) **returns** a solution, or failure
initialize the frontier using the initial state of *problem*
loop do

if the frontier is empty **then return** failure
choose a leaf node and remove it from the frontier
if the node contains a goal state **then return** the corresponding solution
expand the chosen node, adding the resulting nodes to the frontier

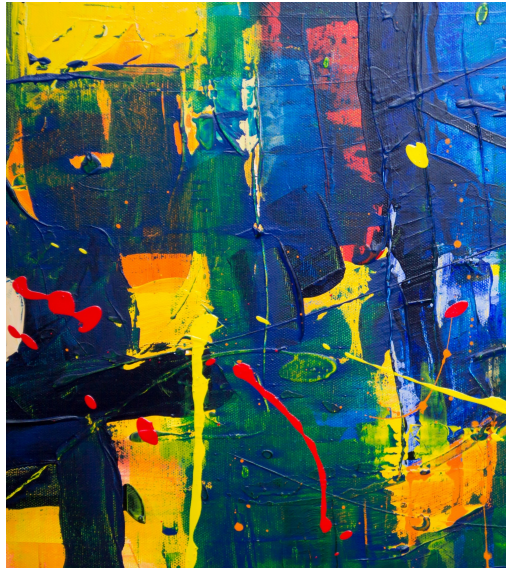
function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
initialize the frontier using the initial state of *problem*
initialize the explored set to be empty
loop do

if the frontier is empty **then return** failure
choose a leaf node and remove it from the frontier
if the node contains a goal state **then return** the corresponding solution
add the node to the explored set
expand the chosen node, adding the resulting nodes to the frontier
only if not in the frontier or explored set

Templates

← não armazena o histórico, portanto permite loops

← armazena o histórico (explored set, closed set), para proibir loops



Artificial Intelligence

Uninformed Search
Ch 3.4

Measuring Problem-Solving Performance

We can evaluate an search algorithm's performance in four ways:

- **Completeness**: Is the algorithm guaranteed to find a solution when there is one?
- **Optimality**: Does the strategy find the optimal solution, i.e., the lowest path cost among all solutions?
- **Time complexity**: How long does it take to find a solution?
- **Space complexity**: How much memory is needed to perform the search?

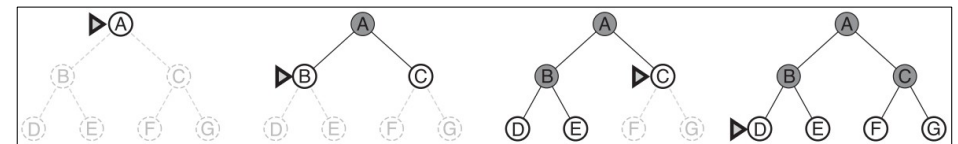
Complexity Measures

- **b**: the **branching factor** or maximum number of successors of any node;
- **d**: the **depth of the shallowest goal node** (i.e., the number of steps along the path from the root);
- **m**: the **maximum length of any path** in the state space.
- **Time** is often measured in terms of the **number of nodes generated during the search**.
- **Space** is often measured in terms of the **maximum number of nodes stored in memory**.

Breadth-first search

- The shallowest unexpanded node is chosen for expansion.
- A FIFO is used for the frontier.
- In general, all the nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded.

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
    node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    frontier ← a FIFO queue with node as the only element
    explored ← an empty set
    loop do
        if EMPTY?(frontier) then return failure
        node ← POP(frontier) /* chooses the shallowest node in frontier */
        add node.STATE to explored
        for each action in problem.ACTIONS(node.STATE) do
            child ← CHILD-NODE(problem, node, action)
            if child.STATE is not in explored or frontier then
                if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
                frontier ← INSERT(child, frontier)
```



Complexity of Breadth-First Search

- **Completeness:** breadth-first search is complete if the branch factor is finite.
- **Optimality:** In general it is not optimal since the costs of the edges are not considered. Optimal if the path cost is a nondecreasing function of the depth of the node. The most common such scenario is that all actions have the same cost.
- **Time Complexity:** Assume a uniform tree where every state has b successors and assume that the solution is at depth d . Then the total number of nodes generated is $b + b^2 + b^3 + \dots + b^d = O(b^d)$.
- **Space complexity:** For breadth-first graph search in particular, every node generated remains in memory. There will be $O(b^{d-1})$ nodes in the explored set and $O(b^d)$ nodes in the frontier, so the space complexity is $O(b^d)$, i.e., it is dominated by the size of the frontier.

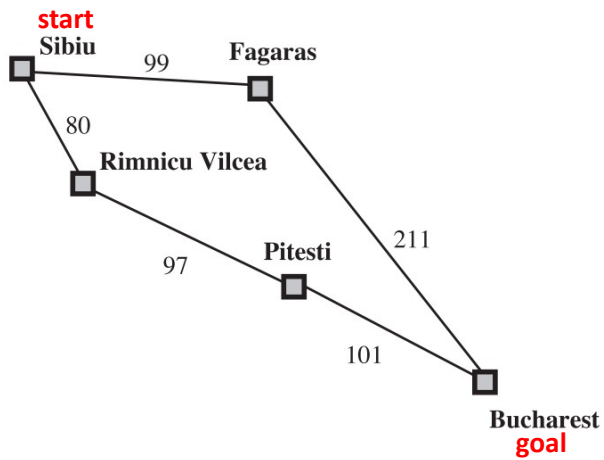
Breadth-first Search

- Always has the **shallowest** path to every node on the frontier.
- Memory requirements are a bigger problem than is execution time.
- **Goal test is applied to each node when it is generated rather than when it is selected for expansion.**
- If the algorithm were to apply the goal test to nodes when selected for expansion, rather than when generated, the whole layer of nodes at depth d would be expanded before the goal was detected and the time complexity would be $O(b^{d+1})$.

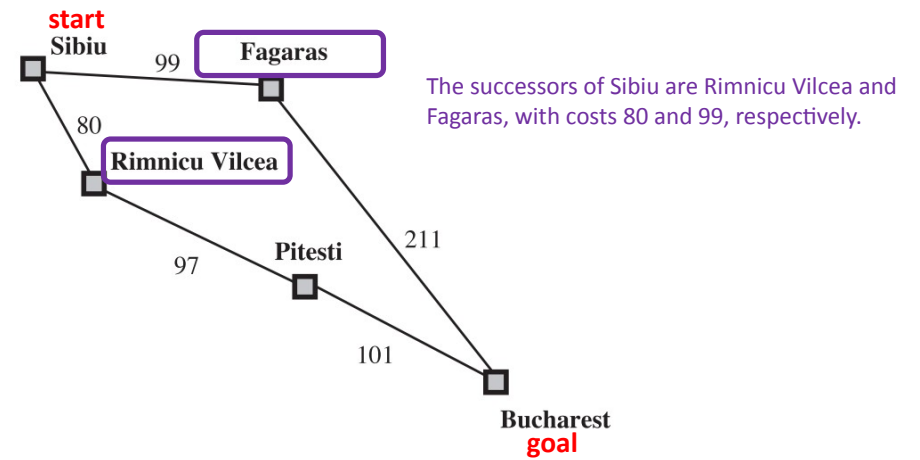
Uniform Cost Search

- Breadth-first search is optimal when all step costs are equal because it always expands the shallowest unexpanded node.
- **Uniform-cost search** expands the node n with the lowest **path cost $g(n)$** .
- The frontier is stored as a priority queue ordered by g .
- **Goal test is applied to a node when it is selected for expansion.**
 - The first **goal node** that is generated may be on a **suboptimal path**.
- A test is added in case a **better path is found** to a node currently on the frontier.

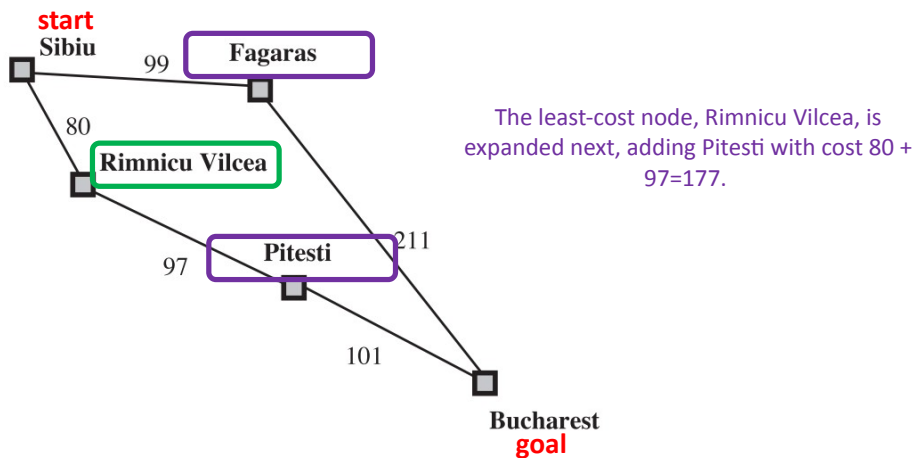
Uniform Cost Search



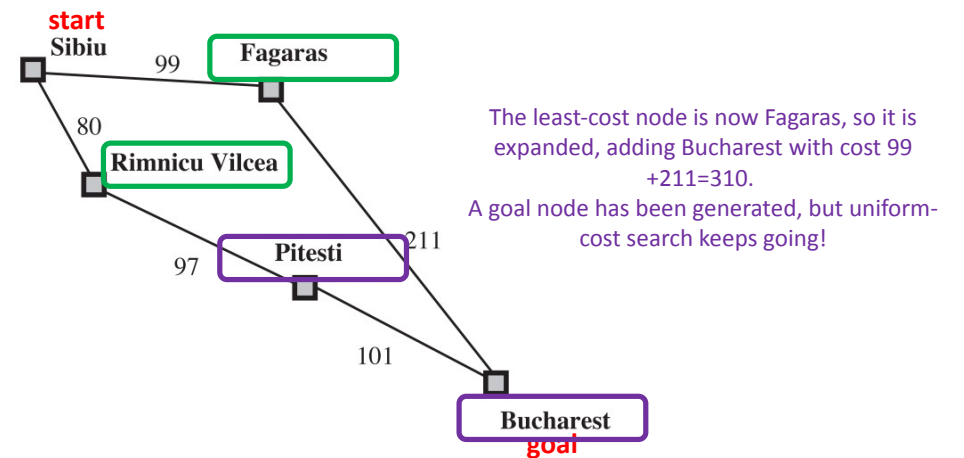
Uniform Cost Search



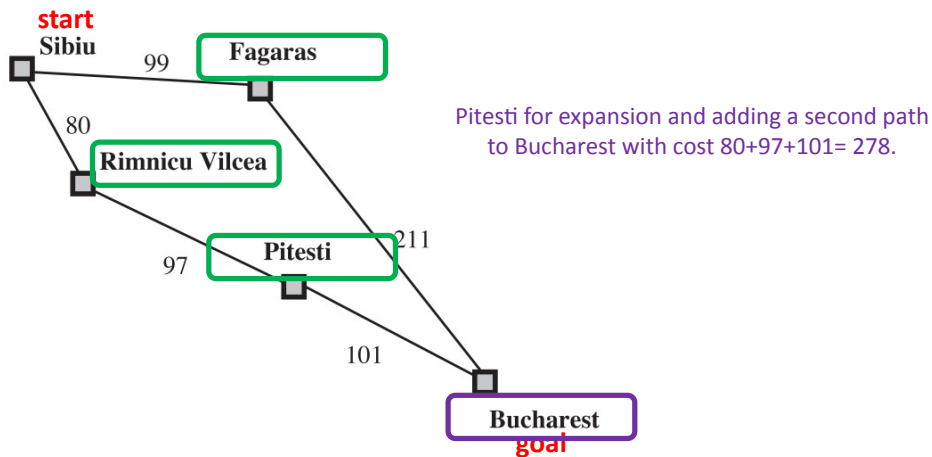
Uniform Cost Search



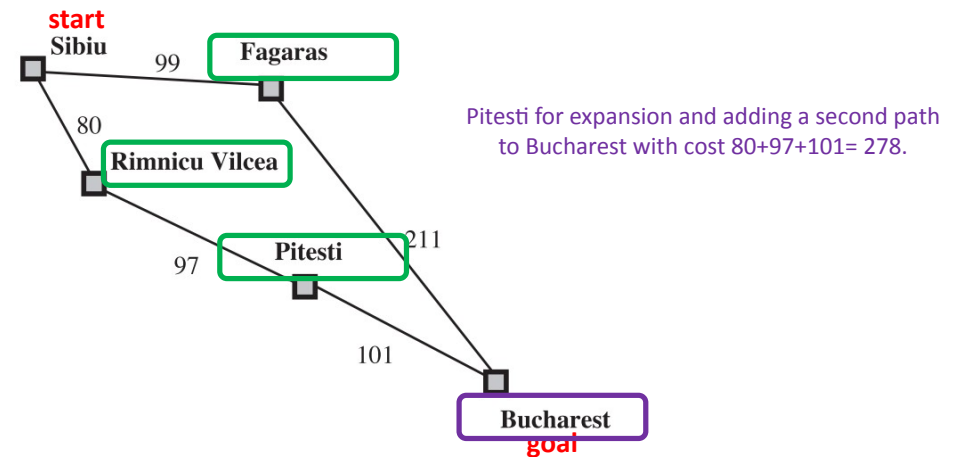
Uniform Cost Search



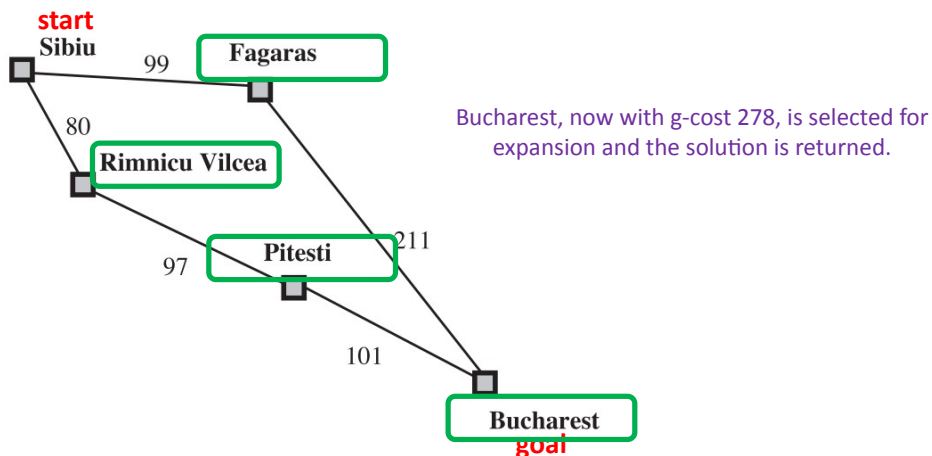
Uniform Cost Search



Uniform Cost Search



Uniform Cost Search

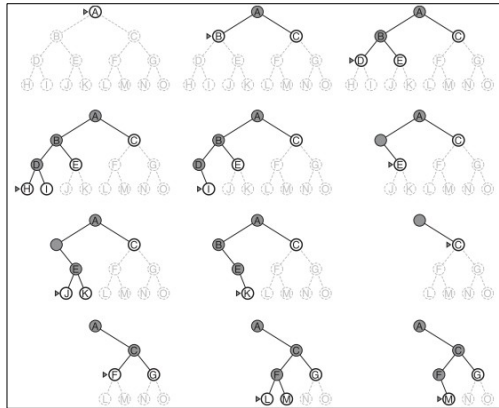


Complexity of Uniform-Cost Search

- **Completeness:** Yes, if the branch factor is finite and all step costs are positive. It will get stuck in an infinite loop if there is a path with an infinite sequence of zero-cost actions (e.g., staying in the same position).
- **Optimality:** **Uniform-cost search is optimal. It expands nodes in order of their optimal path cost. Hence, the first goal node selected for expansion must be the optimal solution.**
- **Time and Space Complexity:** Then the algorithm's worst-case time and space complexity is $O(b^{1+[C^*/\epsilon]})$, which can be much greater than b^d where C^* is the cost of the optimal solution and ϵ is the smallest cost among all actions. When all step costs are equal, $b^{1+[C^*/\epsilon]}$ is just b^{d+1} . When all step costs are the same, uniform-cost search is similar to breadth-first search, except that the latter stops as soon as it generates a goal.

Depth-First Search

- Always expands the deepest node in the current frontier.
- Search proceeds immediately to the deepest level of the search tree, where the nodes have no successors.
- Frontier uses a LIFO queue.
- Can be implemented as GRAPH-SEARCH or with a recursive function that calls itself on each of its children in turn (TREE-SEARCH-style).
 - Can be modified at no extra memory cost so that it checks new states against those on the path to avoid infinite loops.



Complexity of Depth-First Search – 1 (bad news)

- **Completeness:**
 - GRAPH: complete in finite state spaces because it will eventually expand every node.
 - TREE: it is not complete.
- **Optimality:** Depth-first search is not optimal.
 - The first goal found is returned even if there is a best one in the next branch.
- **Time Complexity:**
 - GRAPH: bounded by the size of the state space (which may be infinite).
 - TREE: may generate all of the $O(b^m)$ nodes in the search tree, where m is the maximum depth of any node; this can be much greater than the size of the state space.

Complexity of Depth-First Search – 2 (good news)

- **Space Complexity [advantage in relation to breadth-first search]:**
 - GRAPH: there is no advantage.
 - TREE: we only need to store a single path from the root to a leaf node and the remaining unexpanded sibling nodes.
 - Node can be removed from memory as soon as all its descendants have been fully explored.
 - For a state space with branching factor b and maximum depth m , depth-first search requires storage of only $O(bm)$ nodes.

Tree-search should be used if memory is a limiting factor or if we can assure that repeated states do not happen during search.

Iterative Deepening Depth-First Search

- Gradually increase the depth limit until a goal is found.
- This will occur when the depth limit reaches d , the depth of the **shallowest** goal node.
- Like depth-first search, modest memory requirements: $O(bd)$.
- Like breadth-first search, complete when the branching factor is finite and optimal when the path cost is a nondecreasing function of the depth of the node.

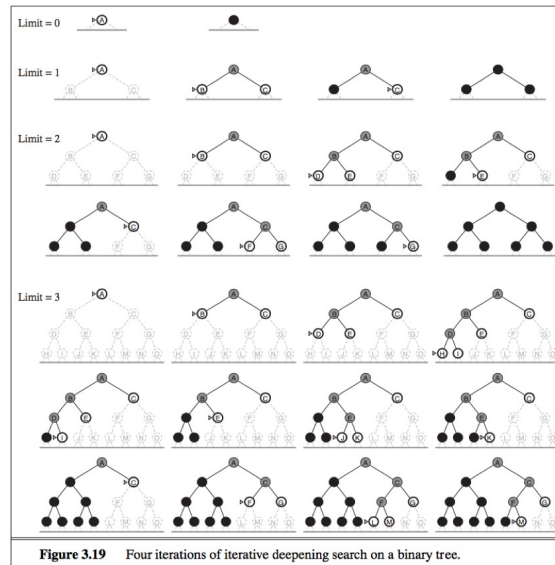


Figure 3.19 Four iterations of iterative deepening search on a binary tree.

Complexity of Iterative Deepening Search

- **Completeness:** it is complete when the branching factor is finite.
- **Optimality:** It is optimal when the path cost is a nondecreasing function of the depth of the node.
- **Time Complexity:** $O(b^d)$ like breadth-first search.
- **Space complexity:** $O(bd)$.

In general, **iterative deepening is the preferred uninformed search method** when the search space is large and the depth of the solution is not known.

Recap

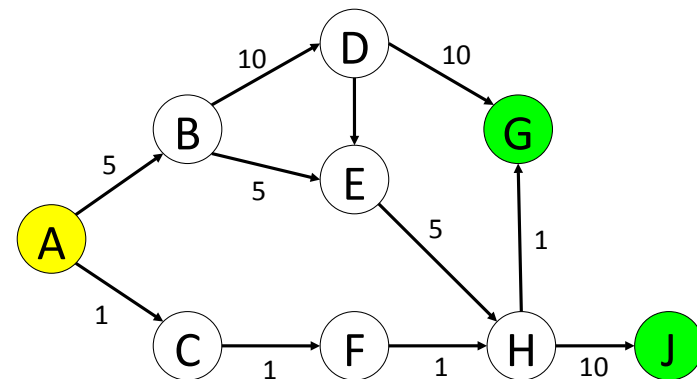
- **Search:** given a problem, find a path (sequence of actions and states that will be visited) that leads from the starting state to the goal state.

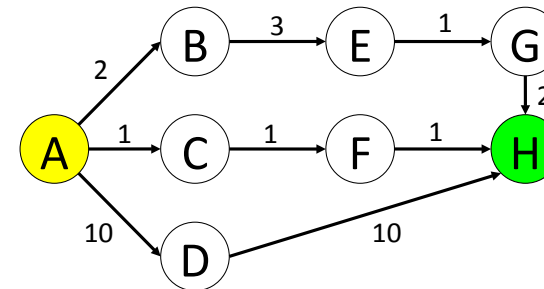
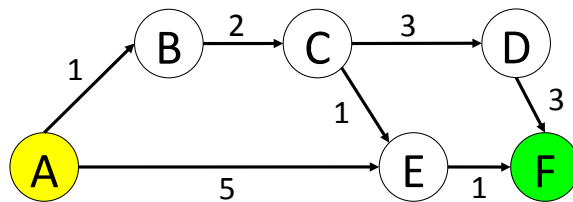
Recap

- **Search:** given a problem, find a path (sequence of actions and states that will be visited) that leads from the starting state to the goal state.
- **Problem specification:** **Initial state**, **valid actions in each state**, **transition model**, **goal test**, **path cost function**.

Recap

- **Search:** given a problem, find a path (sequence of actions and states that will be visited) that leads from the starting state to the goal state.
- **Problem specification:** **Initial state**, **valid actions in each state**, **transition model**, **goal test**, **path cost function**.
- Uninformed search algorithms: [exponential time complexity]
 - Tree-search: low memory requirements, allow loops
 - Graph-Search: high memory requirements, prevent loops
 - Breadth-first search: complete, shallowest path
 - Uniform-cost search: complete, shortest path, high memory requirements
 - Depth-first search (tree version): lower memory requirements, not complete, allow loops.
 - Iterative deepening: low memory requirements, complete, optimal (when the path cost is a nondecreasing function of the depth of the node)





Algorithm 2: Dijkstra's Algorithm for Single-Pair Shortest Path Problem

Data: A weighted directed graph $G = (V, E)$, a source vertex s and a destination vertex d

Result: A list of vertices on the shortest path between s and d

Function DijkstraPair(G, s, d):

```

foreach  $v \in V$  do
     $dist[v] = +\infty$ ;
end
 $dist[s] = 0$ ;
Create an empty vertex set  $Q$ ;
foreach  $v \in V$  do
    Add  $v$  into  $Q$  with key  $dist[v]$ ;
end
while  $Q \neq \emptyset$  do
    Find vertex  $u$  in  $Q$  whose  $dist[u]$  is the smallest;
    Remove  $u$  from  $Q$ ;
    if  $u = d$  then
        break;
    end
     $visited[u] = \text{true}$ ;
    if  $dist[u] == +\infty$  then
        break;
    end
    foreach  $v \in Adj(u)$  and  $visited[v] \neq \text{true}$  do
         $dist = dist[u] + w(u, v)$ ;
        if  $dist < dist[v]$  then
             $dist[v] = dist$ ;
            Update  $Q$  on vertex  $v$  with new  $dist[v]$  value;
             $prev[v] = u$ ;
        end
    end
end
end
return buildPath( $s, d, prev$ )
  
```

Algorithm 5: Uniform-Cost Algorithm for Single-Pair Shortest Path Problem

Data: A weighted directed graph $G = (V, E)$, a source vertex s and a destination vertex d

Result: A list of vertices on the shortest path between s and d

Function UniformCostPair(G, s, d):

```

 $dist[s] = 0$ ;
Create an empty vertex set  $Q$ ;
Add  $s$  into  $Q$  with key  $dist[s]$ ;
while  $Q \neq \emptyset$  do
    Find vertex  $u$  in  $Q$  whose  $dist[u]$  is the smallest;
    Remove  $u$  from  $Q$ ;
    if  $u = d$  then
        break;
    end
    foreach  $v \in Adj(u)$  do
         $dist = dist[u] + w(u, v)$ ;
        if  $v \in Q$  then
            if  $dist < dist[v]$  then
                 $dist[v] = dist$ ;
                 $prev[v] = u$ ;
                Update  $Q$  on vertex  $v$  with new  $dist[v]$  value;
            end
        else
             $dist[v] = dist$ ;
            Add  $v$  into  $Q$  with key  $dist[v]$ ;
             $prev[v] = u$ ;
        end
    end
end
return buildPath( $s, d, prev$ );
  
```

In Dijkstra's algorithm, we initialize Q with all vertices. Therefore, it is only applicable for explicit graphs and has more memory requirements.

algorithms extracted from <https://www.baeldung.com/cs/uniform-cost-search-vs-dijkstras>



Artificial Intelligence

Informed (Heuristic) Search

Ch 3.5

Best-First Search

- Best-first search is an instance of the general TREE-SEARCH or GRAPH-SEARCH algorithm in which a node is selected for expansion based on an **evaluation function $f(n)$** .
- The evaluation function is built as a **cost estimate**, so the node with the lowest evaluation is expanded first.
- Identical to uniform-cost search, except for the use of **f** instead of the partial path cost function **g** to prioritize nodes for expansion.
- The choice of **f** determines the search strategy.

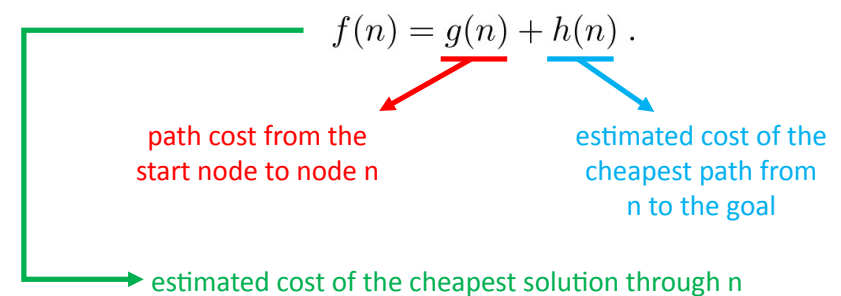
Best-First Search

- Most best-first algorithms include as a component of f a **heuristic function, $h(n)$** , defined as the estimated cost of the **cheapest path from the state at node n to a goal state**.
- For example, in Romania, one might estimate the cost of the cheapest path from Arad to Bucharest via the **straight-line distance** from Arad to Bucharest.
- Heuristic functions are the most common form in which additional knowledge of the problem is imparted to the search algorithm.
- **Heuristics assumption**: nonnegative, problem-specific functions, with one constraint: if n is a goal node, then $h(n)=0$.

Greedy Best-First Search

Greedy best-first search (or greedy search or best-first search) tries to expand the node that is **closest to the goal**, on the grounds that this is likely to lead to a solution quickly. Thus, it evaluates nodes by using just the heuristic function; that is, $f(n) = h(n)$. **Not optimal, but with small search cost.**

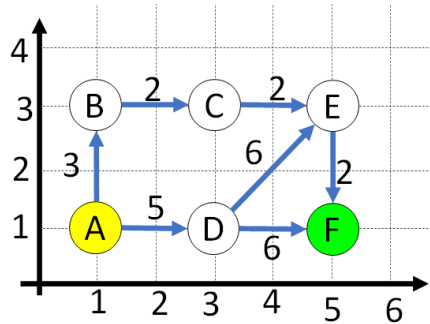
A* Search



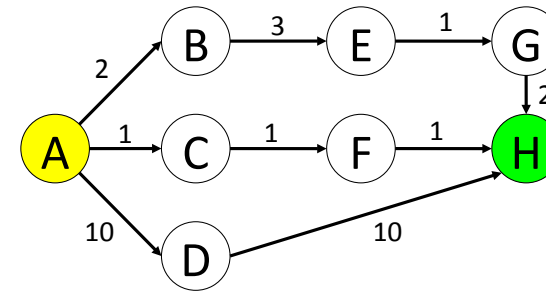
Given some assumptions regarding $h(n)$, A* search is both complete and optimal. The algorithm is identical to UNIFORM-COST-SEARCH except that A* uses $g + h$ instead of g . The value of f is updated whenever its value is reduced.

A* Search Example

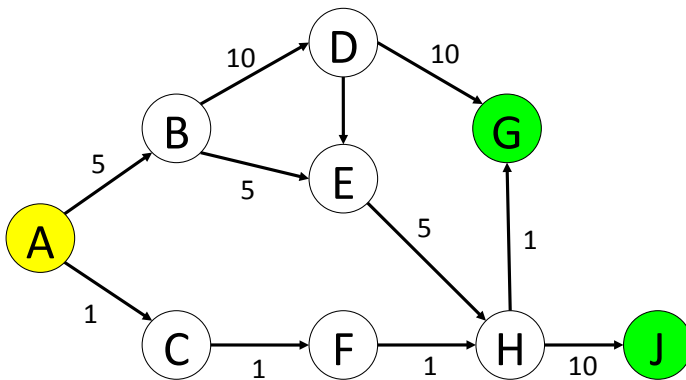
h : nº de quadrados até o destino.



h : nº de nós até o destino



h : nº de nós até o destino



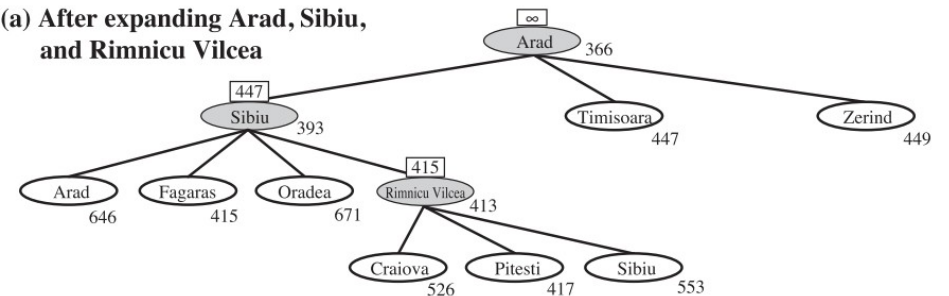
Optimality and the Heuristic Function

- $h(n) = 0$: only $g(n)$ plays a role; A* turns into Dijkstra's Algorithm, which is guaranteed to find a **shortest path**.
- $h(n) \leq \text{cost from } n \text{ to the goal}$: A* is guaranteed to find a shortest path.
 - The lower $h(n)$ is, the more node A* expands, making it slower.
- $h(n) = \text{cost from } n \text{ to the goal}$: A* will only follow the best path and never expand anything else, making it very fast.
 - It's nice to know that given perfect information, A* will behave perfectly.
- $h(n)$ is sometimes greater than the cost from n to the goal:
 - A* is not guaranteed to find a shortest path, but it can run faster.
- $h(n)$ is very high relative to $g(n)$: only $h(n)$ plays a role; A* turns into Greedy Best-First-Search.
- We can decide what we want to get out of A*.
 - With 100% accurate estimates, we'll get shortest paths really quickly.
 - If we're too low, then we'll continue to get shortest paths, but it'll slow down.
 - If we're too high, then we give up shortest paths, but A* will run faster.

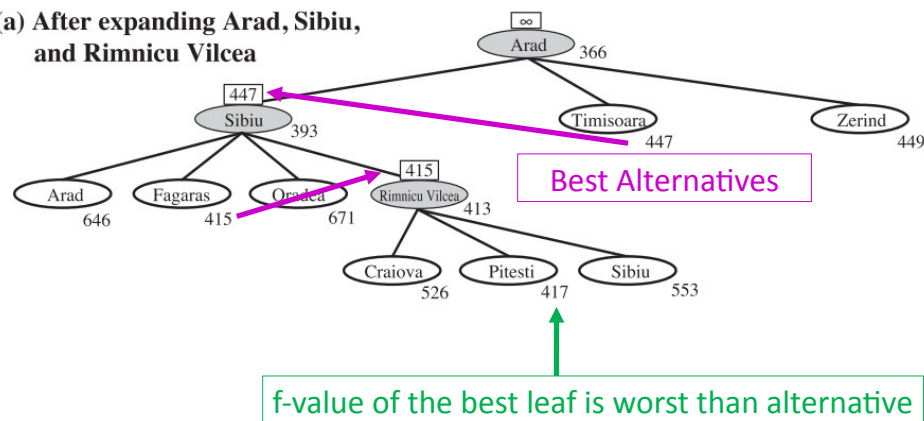
Recursive Best-First Search (RBFS)

- A* stores all generated nodes, therefore it has high memory consumption.
- RBFS reduces the amount of memory used by increasing the need for computing.
- Store the f-value of the **best alternative path** from any **ancestor**.
- If the current node exceeds this limit, unwinds back to the alternative path.
 - The subtree is discarded to reduce the amount of memory needed and rebuilt if necessary.
- RBFS is an optimal algorithm (given the assumptions as A*) and its space complexity is linear instead of exponential in the depth of the deepest optimal solution

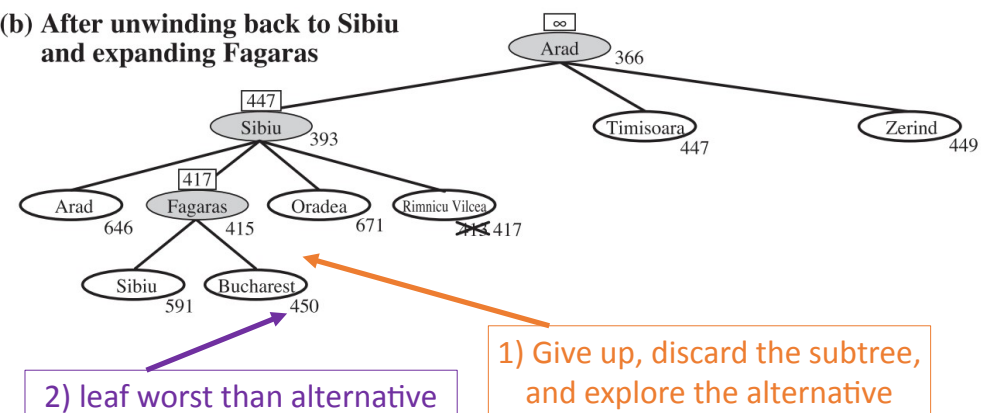
(a) After expanding Arad, Sibiu, and Rimnicu Vilcea



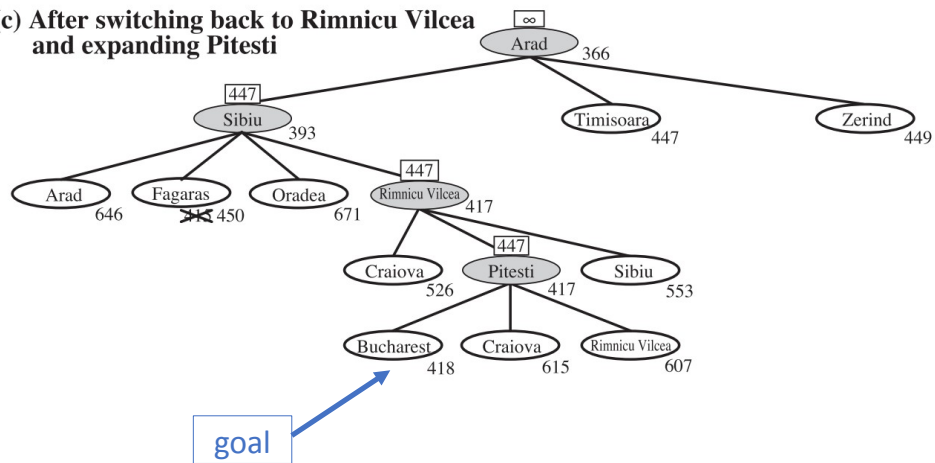
(a) After expanding Arad, Sibiu, and Rimnicu Vilcea



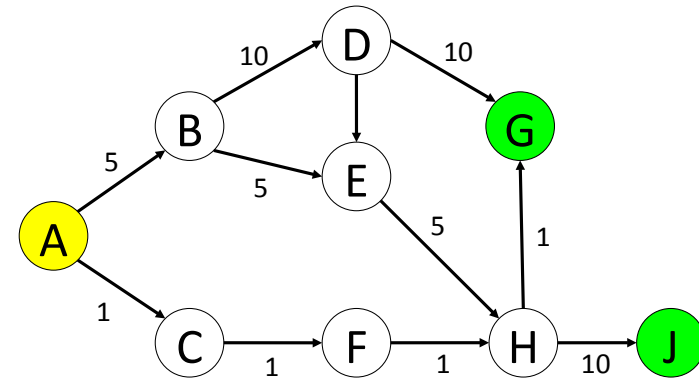
(b) After unwinding back to Sibiu and expanding Fagaras



(c) After switching back to Rimnicu Vilcea and expanding Pitesti



h : nº de nós até o destino

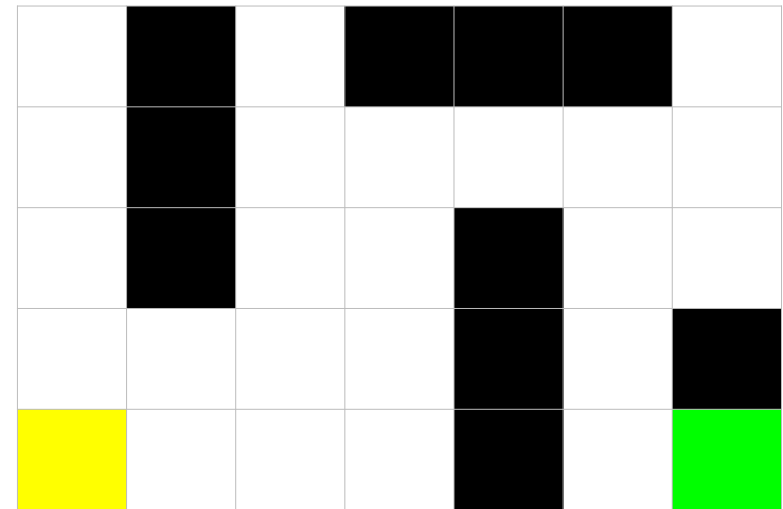
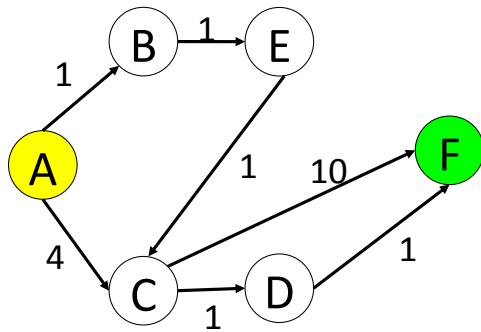
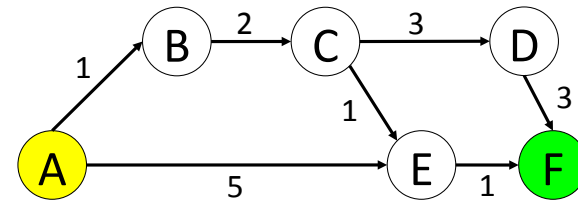


Perguntas?

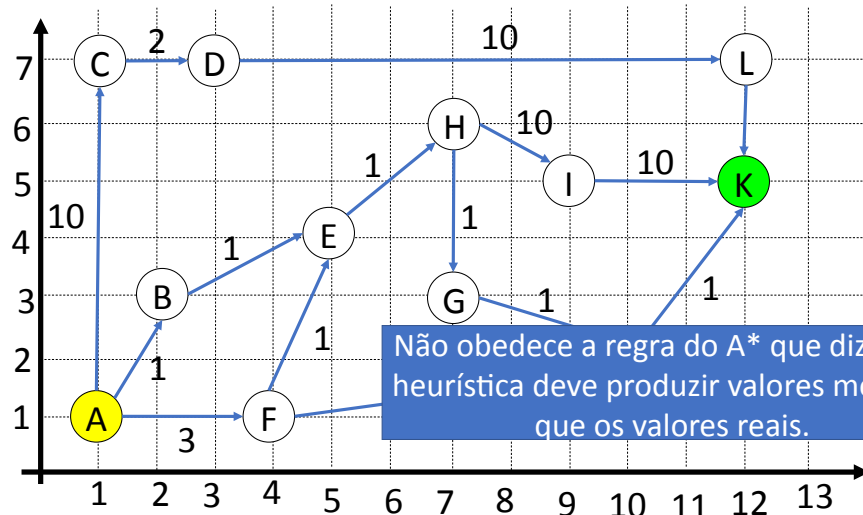
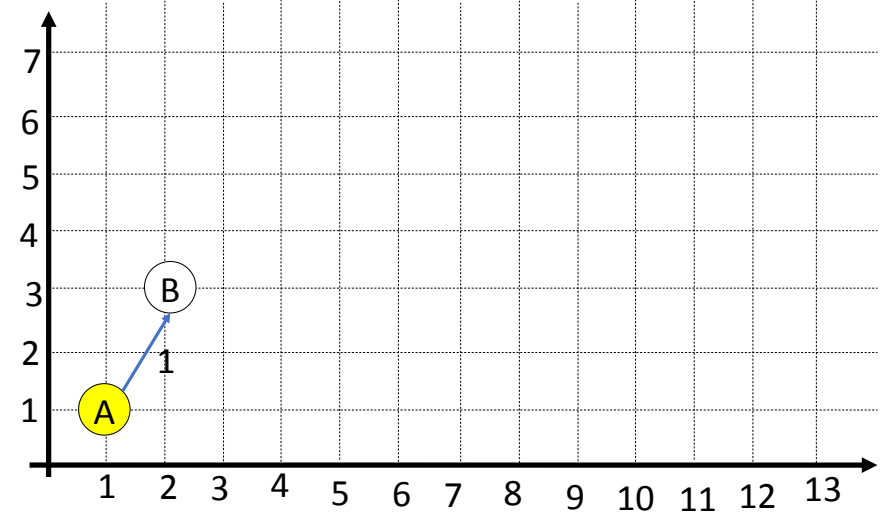
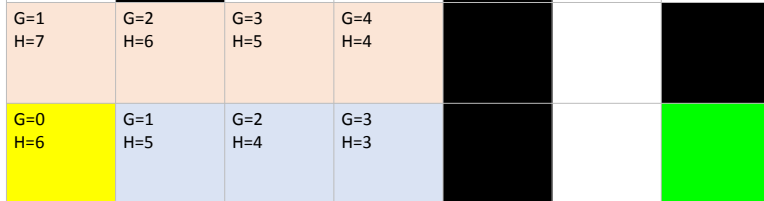


Artificial Intelligence

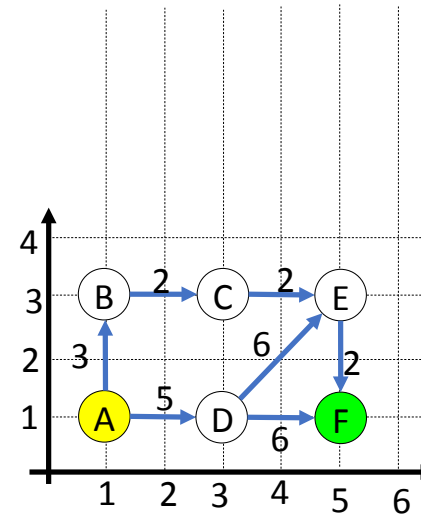
Complexity and Comparison of
Uninformed Search Algorithms
Ch 3.4



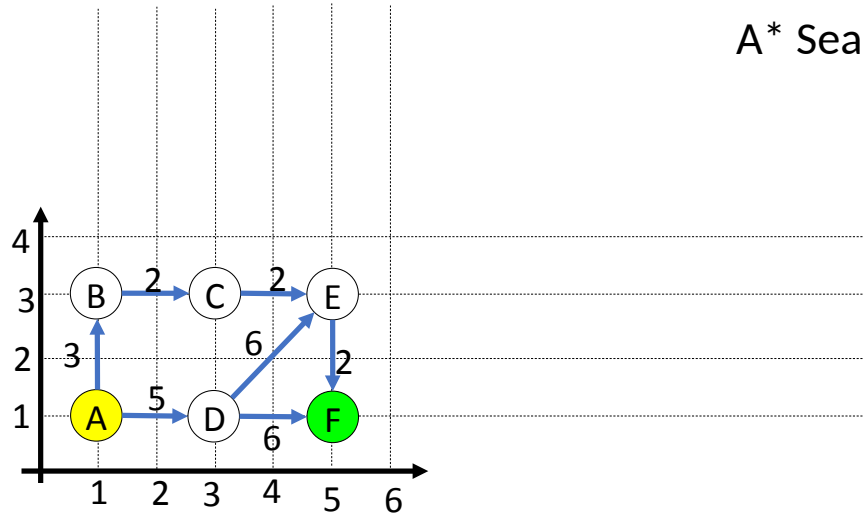
INCOMPLETO



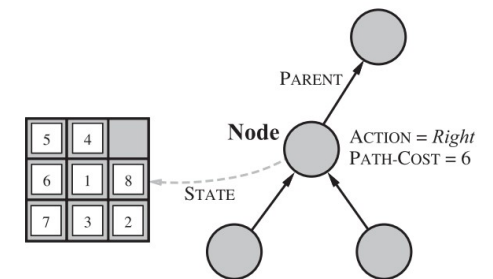
Não obedece a regra do A* que diz que a heurística deve produzir valores menores que os valores reais.



A* Search



Node Data Structure



For each node n of the tree, we have a structure that contains four components:

- **$n.STATE$** : the state in the state space to which the node corresponds
- **$n.PARENT$** : the node in the search tree that generated this node;
- **$n.ACTION$** : the action that was applied to the parent to generate the node;
- **$n.PATH-COST$** : the cost, traditionally denoted by $g(n)$, of the path **from the initial state to the node**, as indicated by the parent pointers.

Implementation Issues

- The node structure allows the extraction of the solution path when a goal node is found by **following parent pointers back to the root** and returning the sequence of actions.
- Notice how the PARENT pointers string the nodes together into a tree structure.
- **States != Nodes**: note that two different nodes can contain **the same world state** if that state is generated via two different search paths.
- The **frontier** are stored in a **queue** so that the search algorithm can easily choose the next node (pop) to expand according to its preferred strategy. Either LIFO, FIFO, or Priority Queues can be used depending on the algorithm.

Additional Implementation Issues

- The **explored set** can be implemented with a **hash table** to allow efficient (nearly constant time insertion and lookup) checking for repeated states.
- One must take care to implement the hash table with the right notion of **equality** between states. For instance, in TSP, the set of visited cities {Bucharest,Urziceni,Vaslui} is the same as {Urziceni,Vaslui,Bucharest}.
- Data structures for states can be built in some **canonical form** in which logically equivalent states map to the same data structure.
- In the case of states described by sets, for example, a bit-vector representation or a sorted list without repetition would be canonical, whereas an unsorted list would not.

Measuring Problem-Solving Performance

- To assess the effectiveness of a search algorithm, we can consider just the **search cost** or we can use the **total cost**, which combines the search cost and the path cost of the solution found.
- **Example**: If driving a car for a long path is faster than searching for a shorter path and then driving, the longer path should be chosen since it is more efficient.